

Statistical Drift Detection in Clustered Dynamic Ensembles

Michał Iwicki, Dawid Sroczyk, Marta Szuwarska

June 8, 2026

1 Introduction

Data stream classification requires models that can operate in continuously evolving environments, where the underlying data distribution may change over time. This phenomenon, known as concept drift, poses a significant challenge, as it can quickly degrade model performance if not properly handled.

Most existing approaches address drift at a global level, assuming that changes affect the entire feature space uniformly. However, in many real-world scenarios, drift is often localized and affects only specific regions of the data.

In this project, we investigate a localized approach to drift handling by combining Cluster-Based Dynamic Ensemble Selection (C-DES) with a distribution-based drift detection method (SDDM). The proposed framework enables region-specific adaptation by detecting drift within clusters and updating model competencies accordingly, while preserving previously learned knowledge.

The goal of this work is to evaluate whether such localized drift detection and adaptation can improve performance and stability in non-stationary data streams.

2 Methodology

This section presents the proposed approach for handling concept drift in data streams. The method combines localized ensemble selection with statistical drift detection to enable region-specific adaptation.

First, we describe the Statistical Drift Detection Method (SDDM), which monitors changes in the data distribution. Next, we introduce the Cluster-Based Dynamic Ensemble Selection (C-DES) framework, which performs localized model selection using clustering. Finally, we present the integration of both components, where drift detection is applied independently within each cluster to guide adaptive behavior of the ensemble.

2.1 Statistical Drift Detection Method (SDDM)

The Statistical Drift Detection Method (SDDM) is a distribution-based approach for detecting concept drift by directly monitoring changes in the joint distribution $P(X, Y)$, without relying on classifier feedback [1]. This makes the method applicable in scenarios where labels may be delayed or unavailable.

Overview. SDDM operates by comparing two empirical distributions:

- a *reference window* representing past data,
- a *current window* representing recent observations.

Drift is quantified as a statistical distance between these distributions.

Data Representation. Continuous features are discretized into a fixed number of bins [1]. To efficiently incorporate nominal features, the algorithm infers feature types during the initialization phase. For any detected nominal feature, dynamic min-max scaling is bypassed. Instead, an incremental lookup dictionary is maintained, mapping each distinct categorical value to a specific numeric bin index. An independent histogram is maintained for each feature. Therefore, for a stream containing m features, the detector maintains m separate feature-wise distributions. Histograms are additionally conditioned on class labels to approximate the joint distribution $P(X, Y)$. Crucially, the historical reference data is represented exclusively through aggregated histograms (H_{ref}) rather than raw instances, minimizing memory overhead.

Two Operation Modes & Memory Optimization. Our implementation supports two variants of SDDM, which differ in how the reference distribution is maintained:

(1) **Sliding Window SDDM (non-incremental).** The current window ($\mathcal{W}_{cur}$) maintains fixed size and holds raw observations. To ensure memory

efficiency, the reference baseline operates strictly on aggregated histogram counts (H_{ref}) and does not store raw observations. To enable the sliding mechanism without raw data, a light-weight FIFO queue (Q_{ref}) stores only the computed bin indices and class labels. As new data arrives, the oldest indices are popped from Q_{ref} and their corresponding counts are removed from H_{ref} .

(2) Incremental SDDM. The reference distribution grows over time and accumulates all past observations (no removal) by continuously incrementing the histogram H_{ref} . No raw observation history or index queues (Q_{ref}) are preserved, enforcing $\mathcal{O}(1)$ memory requirements for the reference baseline. This results in a more stable estimate of the historical distribution, but makes the detector more sensitive to gradual deviations from long-term behavior.

The current window $\mathcal{W}_{cur}$ of size p is maintained in both cases and always represents the most recent data.

Window Update Mechanism. At each time step:

1. The new instance is added to $\mathcal{W}_{cur}$ and its corresponding feature-wise histograms (H_{cur}) are updated.
2. If $\mathcal{W}_{cur}$ is full, the oldest raw element shifts out, and its corresponding bin indices update the reference histogram H_{ref} .
3. In the non-incremental mode, these bin indices are also pushed to the queue Q_{ref} . If Q_{ref} exceeds the reference size, the oldest indices are popped, and their counts are subtracted from H_{ref} .

Drift Quantification. For each feature, the divergence between the distributions in $\mathcal{W}_{cur}$ and $\mathcal{W}_{ref}$ is computed using symmetric Kullback–Leibler divergence:

$$D_{KL}(P||Q) = \frac{1}{2} \left(\sum_i P_i \log \frac{P_i}{Q_i} + \sum_i Q_i \log \frac{Q_i}{P_i} \right)$$

The overall drift magnitude is defined as:

$$D = \max_f D_{KL}^{(f)}$$

Drift Detection Criterion. A drift is detected when:

$$D > \tau$$

where τ is a predefined threshold.

Drift Localization. The source of drift is identified as:

$$f^* = \arg \max_f D_{KL}^{(f)}$$

which provides interpretability by indicating the most affected feature.

Warm-up Phase & Limit Initialization.

To ensure the statistical robustness of the discretization limits, both the initial stream warm-up and the post-drift recovery phases utilize a temporary initialization buffer ($\mathcal{B}_{init}$). Incoming stream observations are accumulated into this buffer until it reaches the capacity of the reference window ($|\mathcal{W}_{ref}|$). Once this statistically significant sample is gathered, the algorithm establishes the global min-max boundaries for continuous features and builds the categorical mapping dictionaries for nominal features. Deriving these limits from a larger sample prevents clamping, which could easily occur if limits were inferred solely from the smaller current window ($\mathcal{W}_{cur}$). After the discrete bins are securely defined and frozen, the buffered instances are mapped to populate the initial reference baseline (H_{ref}). The temporary buffer $\mathcal{B}_{init}$ is then flushed, and the sliding current window ($\mathcal{W}_{cur}$) begins gathering fresh stream data from scratch.

Reset Strategy and Re-binning. When a concept drift is triggered, a localized reconstruction phase begins. To accelerate recovery while retaining information about the newly emerging concept, the detector does not discard recent observations. Instead, the raw data points currently residing in $\mathcal{W}_{cur}$ are immediately transferred into the temporary initialization buffer ($\mathcal{B}_{init}$). Historic data, active histograms, shift queues, and previously established discrete boundaries (min-max limits and nominal dictionaries) are entirely cleared. The global boundaries are initially recalculated using the transferred instances in $\mathcal{B}_{init}$. The algorithm then resumes the warm-up phase, strictly suspending statistical testing until the buffer collects enough subsequent stream samples to reach the full capacity of $|\mathcal{W}_{ref}|$.

Discussion of Modes. The two modes introduce a trade-off:

- Sliding window mode reacts faster to recent changes and limits drift propagation.
- Incremental mode provides a stable long-term baseline but may over-estimate drift magnitude due to accumulation of historical data.

Algorithm 1: [SDDM Update Cycle](#)

Input: New instance (x, y) , Temp buffer $\mathcal{B}_{init}$, Current window $\mathcal{W}_{cur}$, Histograms H_{cur}, H_{ref} , Shift queue Q_{ref}

/ Phase 1: Warm-up / Re-binning */*

- 1 **if** *limits are not set* **then**
- 2 Add (x, y) to $\mathcal{B}_{init}$;
- 3 **if** $\text{length}(\mathcal{B}_{init}) \geq \text{ref_window_size}$ **then**
- 4 Establish global min-max limits and nominal maps using all instances in $\mathcal{B}_{init}$;
- 5 **for each** (x_{old}, y_{old}) in $\mathcal{B}_{init}$ **do**
- 6 Compute bin indices B_{old} for x_{old} using established limits;
- 7 Update H_{ref} by adding counts at B_{old} given y_{old} ;
- 8 **if not incremental** **then**
- 9 Push (y_{old}, B_{old}) to Q_{ref} ;
- 10 Clear $\mathcal{B}_{init}$, $\mathcal{W}_{cur}$, and H_{cur} ;
- 11 Mark limits as **set**;
- 12 **return**;

/ Phase 2: Normal Operation Mode */*

- 13 Add (x, y) to $\mathcal{W}_{cur}$ and update H_{cur} ;
- 14 **if** $\mathcal{W}_{cur}$ is full **then**
- 15 Pop oldest raw element (x_{old}, y_{old}) from $\mathcal{W}_{cur}$;
- 16 Update H_{cur} by subtracting (x_{old}, y_{old}) ;
- 17 Compute bin indices B_{old} for x_{old} ;
- 18 Update H_{ref} by adding counts at B_{old} given y_{old} ;
- 19 **if not incremental** **then**
- 20 Push (y_{old}, B_{old}) to Q_{ref} ;
- 21 **if** $\text{length}(Q_{ref}) > \text{ref_window_size}$ **then**
- 22 Pop oldest (y_{pop}, B_{pop}) from Q_{ref} ;
- 23 Update H_{ref} by subtracting counts at B_{pop} given y_{pop} ;

/ Phase 3: Statistical Testing */*

- 24 **if** both windows are full **and** time to test **then**
- 25 **for each** feature f **do**
- 26 Compute $D^{(f)} = D_{KL}(H_{cur}^{(f)} || H_{ref}^{(f)})$;
- 27 $D \leftarrow \max_f D^{(f)}$;
- 28 **if** $D > \tau$ **then**
- 29 */* Moving current data to reference and starting warm-up again */*
- 30 Mark limits as **not set**;
- 31 Clear previous global min-max limits and nominal dictionaries;
- 32 Transfer all elements from $\mathcal{W}_{cur}$ to $\mathcal{B}_{init}$;
- 33 Clear $\mathcal{W}_{cur}$, Q_{ref} , H_{cur} , and H_{ref} ;

Estimation of Mean and Standard Deviation To distinguish major from minor drift, the detector additionally estimates the mean and standard deviation of the observed drift magnitudes. Since statistical testing is performed only at fixed intervals (default = 10), these statistics are updated exclusively using magnitudes measured outside the current post-drift adaptation phase, preventing temporary magnitude inflation immediately after drift detection. Furthermore, a fixed-length (default = 50) queue is maintained to ensure consistent statistical properties and comparable drift scaling over time.

Updates Compared to the Preliminary Report

Several improvements were introduced compared to the preliminary report.

Support for nominal features was added through dedicated categorical mappings, together with a clearer description of the histogram organization and feature-wise distributions.

The memory management strategy was redesigned. In incremental mode, the reference distribution now stores only aggregated histograms instead of raw observations, preventing unbounded memory growth. In sliding-window mode, only lightweight queues of discretized bin indices are additionally maintained to enable efficient removal of outdated histogram contributions.

The warm-up and post-drift recovery procedures were unified into a consistent re-binning mechanism based on a temporary initialization buffer. This prevents reuse of outdated discretization limits after drift detection.

Additionally, estimation of the mean and standard deviation of drift magnitudes was introduced to normalize measured drift strength and improve distinction between minor and major distribution changes.

Finally, the algorithm description and pseudocode were rewritten to better align with the actual implementation and to clarify the detector workflow and reset procedure.

2.2 Cluster-Based Dynamic Ensemble Selection (C-DES)

The proposed Cluster-Based Dynamic Ensemble Selection (C-DES) framework is implemented by extending the Streaming Random Patches (SRP) ensemble provided by the River library. While the original SRP implementation utilizes global performance metrics for model weighting, our modified version introduces a clustering layer to facilitate localized ensemble selection and region-specific drift adaptation. This shifts the ensemble’s logic from global averaging to localized, dynamic competence tracking, allowing the system to identify which base learners are most capable of handling specific data distributions within a non-stationary stream [2].

This localization is achieved through a context mapping phase where arriving instances are processed by a KMeans clustering algorithm to determine their active region. This ensures that the ensemble’s behavior is tailored to the specific "context" of the incoming data rather than treating the entire stream as a homogeneous space. Competency within the ensemble is managed through a strict test-then-train protocol tailored to these discovered clusters. Before a base learner is updated with a new instance, its prediction is evaluated against the true label, and its specific competency score for the active cluster is updated accordingly. In our implementation, this is handled via a dictionary of metrics within each learner, which allows for the automatic initialization of unique performance trackers as new clusters emerge in the stream.

During the prediction phase, the algorithm employs a "Better-than-Random" deactivation strategy to filter the ensemble. For any given instance, the system calculates a threshold based on the number of observed classes, typically defined as $1/n$ for an n -class problem, performance of a majority class baseline for the active cluster. Specifically, a model must achieve an accuracy higher than simply predicting the most frequent class within that local region to be considered competent. Any base learner whose localized competency score falls below this threshold for the current cluster is assigned a voting weight of zero. In the event that all base learners are found incompetent (meaning no model outperforms the local majority class baseline) the ensemble defaults to predicting the cluster’s true majority class. This ensures that the system falls back to the smartest naive prediction rather than generating an empty or random guess. This dynamic selection minimizes noise by silencing "incompetent" models that have not yet mastered the current region or have been rendered obsolete by localized drift, effectively transitioning the ensemble from dynamic weighting to true selection [2].

Furthermore, in dynamic data streams, the spatial boundaries of clusters can evolve over time as new data arrives. To account for this structural drift and its impact on localized competency scores, the ensemble does not track performance metrics indefinitely. Instead, competency is measured using a rolling window (the last 100 instances). This allows the competency scores to smoothly adapt to moving cluster boundaries and seamlessly "forget" historical performance data that may belong to outdated spatial distributions.

A critical innovation of C-DES is its "Soft Reset" mechanism for handling detected concept drifts. Unlike standard ensemble methods that perform a global hard reset by destroying and replacing models upon drift detection, C-DES preserves the underlying base learners and their historical knowledge.

When a drift is signaled within a specific region, only the competency metrics associated with that cluster are reset. This forces the base learners to "re-earn" their voting rights for that region while maintaining their expertise in stable clusters. This approach facilitates significantly faster recovery times and maintains higher overall stability, particularly in scenarios where old concepts may recur.

The core logic of the C-DES enhanced Streaming Random Patches ensemble is summarized in Algorithm 1. This implementation integrates localized context mapping via KMeans with a dynamic selection threshold to ensure model competence in non-stationary environments.

```

1 Algorithm 1: C-DES Learn and Predict Cycle
2 # Input: Instance x, True label y, Ensemble E, Clusterer
  C
3 # Output: Prediction y_hat
4
5 1. Context Mapping:
6   C.learn_one(x)
7   cluster_id = C.predict_one(x)
8   Update local class counts for cluster_id
9
10 2. Predict Phase (Dynamic Selection):
11   y_pred_ensemble = Counter()
12   threshold = 1.0 / num_classes
13   threshold = max(local_class_counts) / total_local_counts
14   weights_sum = 0.0
15
16   for model in E:
17       y_proba = model.predict_proba_one(x)
18       # Retrieve localized accuracy for the active
        region
19       acc_local = model.cluster_metrics[cluster_id].get
        ()
20
21       # Selection Rule: Better-than-Random Deactivation
22       weight = acc_local if acc_local > threshold else
        0.0
23       y_pred_ensemble.update({k: v * weight for k, v in
        y_proba.items()})
24       weights_sum += weight
25
26   if weights_sum == 0.0:
27       # Fallback: All models are incompetent
28       y_hat = majority_class(cluster_id)

```



```

29     else:
30         y_hat = max(y_pred_ensemble, key=y_pred_ensemble.
31                     get)
32
33 3. Competency Scoring (Test-then-Train):
34     for model in E:
35         y_pred_model = model.predict_one(x)
36         if y_pred_model is not None:
37             # Update local metric before training
38             model.cluster_metrics[cluster_id].update(y,
39             y_pred_model)
40
41             # Train base learner with subspace x_subset
42             model.learn_one(x_subset, y)
43
44 4. Soft Reset (Drift Detection):
45     if model.drift_detector.drift_detected:
46         # Reset only the active region's competence
47         score
48         model.cluster_metrics[cluster_id].reset()

```

Listing 1: Cluster-Based Dynamic Ensemble Selection (C-DES)

Updates Compared to the Preliminary Report

Several key improvements were made to the C-DES framework compared to the preliminary report.

Firstly, to better handle imbalanced data, the static deactivation threshold (previously $1/n$) was replaced; the system now considers the option of silencing members worse than the performance of the majority class baseline for the active cluster.

Secondly, a strict fallback mechanism was implemented: in the event that all base learners are found incompetent, the ensemble safely defaults to predicting the local majority class.

Finally, acknowledging that clusters can evolve over time, localized competency tracking was updated to use a rolling window (100 instances). This prevents indefinite performance tracking, allowing localized competency scores to smoothly adapt to moving cluster boundaries and structural drift.

2.3 Combining C-DES with SDDM

The proposed approach integrates the data-driven drift detection capability of SDDM with the localized ensemble selection mechanism of C-DES. The

key motivation is that concept drift in data streams often occurs only within specific regions of the feature space. In such cases, global drift detection or global model adaptation may be inefficient or even misleading.

To address this, we assign an independent SDDM detector to each cluster identified by the C-DES framework. Each detector monitors the local data distribution within its corresponding cluster. ~~When a drift is detected in a specific region, only the models' competencies associated with that cluster are reset.~~

~~More precisely, upon drift detection:~~

- ~~• the local competency metrics (weights) of all base learners for the affected cluster are reset,~~
- ~~• the underlying models are preserved,~~
- ~~• the models must re-establish their competence in the new concept.~~

When a drift is detected in a specific region, the system analyzes the magnitude of the divergence reported by the SDDM detector to distinguish between minor and major drifts, applying an appropriate reset strategy for each:

- **Minor Drift:** If the drift magnitude falls within standard expected deviations, a *Soft Reset* is triggered. The local competency metrics (weights) of all base learners for the affected cluster are cleared, but the underlying models and their structures are preserved. The models must continuously learn and re-establish their competence in the new concept.
- **Major Drift:** If the drift magnitude is statistically severe—specifically, if it exceeds the historical mean magnitude plus a predefined number of standard deviations (e.g., $\mu + 3.0\sigma$)—a *Hard Reset* is triggered. The underlying base learners are completely reset and replaced, allowing the ensemble to rapidly adapt to severe, permanent conceptual shifts without being constrained by outdated historical trees.

This design aligns with the "soft reset" philosophy of C-DES, but enhances it with explicit, distribution-based drift detection. As a result, the system can react quickly to localized changes without disrupting stable regions of the data stream.

~~We consider two complementary adaptation mechanisms for individual base learners:~~

- ~~**Passive adaptation:** models gradually adapt through continuous learning,~~
- ~~**Active adaptation:** each model may additionally employ an internal drift detector (e.g., ADWIN) to monitor its predictive performance.~~

To systematically evaluate the impact of localized selection and data-driven drift detection, our framework is designed to support four distinct structural configurations:

- **Vanilla SRP:** The standard Streaming Random Patches baseline. It uses global ensemble voting and relies entirely on error-driven ADWIN detectors to trigger hard resets (complete tree replacement) when classification performance degrades.
- **Base C-DES:** The foundational C-DES baseline. It introduces context mapping and dynamic model deactivation. While it disables the global hard resets of the underlying SRP base learners, it employs an independent ADWIN detector for each cluster to monitor the ensemble’s localized predictive error. If this local ADWIN registers a significant drop in regional accuracy, it triggers a "soft reset" by clearing the competency metrics for that specific cluster.
- **C-DES-SDDM:** Integrates SDDM for data-driven, localized drift detection. Minor distributional shifts trigger a "soft reset" (clearing only the localized competency metrics for the affected cluster). Because internal ADWIN detectors are disabled, SDDM is also granted authority to trigger structural hard resets if the drift magnitude is statistically severe (exceeding a dynamic threshold based on the historical mean and standard deviation of drift magnitudes).
- **C-DES-Hybrid:** A hybrid approach where both detection mechanisms operate simultaneously. SDDM monitors the data distribution to trigger region-specific soft resets, while the internal ADWIN detectors remain enabled to actively monitor the predictive error of individual base learners. If a learner’s accuracy drops significantly, ADWIN overrides and executes a structural hard reset.

In this work, SDDM operates in its sliding window configuration to ensure fast responsiveness to local distribution changes. This choice prioritizes rapid adaptation over long-term stability, which is crucial in highly dynamic streaming environments.

Updates Compared to the Preliminary Report

Compared to the preliminary report, the integration and evaluation of our methods were significantly refined. We introduced standardized acronyms for each method configuration (Baseline SRP, Base C-DES, C-DES-SDDM, and C-DES-Hybrid) to ensure consistency throughout the text. We also clarified the exact mechanisms of each: for example, clarifying that the Base C-DES actually utilizes localized ADWIN detectors to trigger soft resets upon regional error degradation, whereas the C-DES-Hybrid’s ADWIN component relies on the standard SRP approach for base learner replacement (i.e., drift warning → background model training → drift confirmation → model replacement).

Furthermore, we improved the overall method by formally distinguishing between major drift (justifying the complete replacement of base learners via a Hard Reset) and minor drift (in which weight updates via a Soft Reset are sufficient). This distinction is now dynamically governed by a statistical evaluation of the SDDM drift magnitude.

3 Experimental Setup

To comprehensively evaluate the Cluster-Based Dynamic Ensemble Selection (C-DES) framework and its proposed drift-handling extensions, we conducted a series of ten experiments. The evaluation initially focuses on establishing the performance of four core configurations: the baseline Streaming Random Patches (SRP) algorithm, the unmodified C-DES framework, C-DES enhanced with SDDM for data-driven weight resets (Passive Adaptation), and C-DES combining SDDM with ADWIN for model-level tree replacement (Adaptation). Building upon these core baseline comparisons, subsequent experiments explore the internal mechanics and broader applicability of the system. Through this progression, we aim to quantify the system’s capacity to detect localized drifts, handle recurrent concepts, and maintain overall computational efficiency in highly dynamic environments.

3.1 Data Streams

For the evaluation of the C-DES and SDDM frameworks, we utilize a combination of custom-designed synthetic generators and real-world benchmark datasets. These are categorized based on the specific challenges they present to the ensemble:

- **Regional and Sensor Drift:** This category focuses on localized

changes within the feature space.

- *RegionalDriftStream* (Exp. 1): Uses the *SEA* generator to create two distinct regions; Region A remains stable while Region B undergoes periodic concept transitions between different variants.
 - *SensorFailureStream* (Exp. 2): Simulates a contextual failure where the decision logic (sensor summation) inverts only within a "High Temperature" region after a specific timestamp.
 - *NoiseQuarantineStream* (Exp. 10): Features three regions where two remain stable and the third transitions into pure random noise, testing the framework’s ability to "quarantine" contaminated clusters without affecting global accuracy.
- **Temporal Dynamics:** These streams test the memory and reaction speed of the ensemble.
 - *RecurrentDriftStream* (Exp. 3): Implements an $A \rightarrow B \rightarrow A$ transition using *Agrawal* classification functions to evaluate if the ensemble can re-activate historical competency when a concept recurs.
 - *RapidFlickerStream* (Exp. 4): Introduces high-frequency concept switching every 250 samples to test the responsiveness of the localized competency trackers.
 - **High-Dimensional Noise:**
 - *NoisyAgrawalStream* (Exp. 6): Injects up to 100 irrelevant, uniformly distributed noise features into a standard *Agrawal* stream to assess the robustness of the KMeans clustering layer against the "curse of dimensionality."
 - **Complex and Conflicting Contexts:**
 - *MultiContextStream* (Exp. 5): Consists of four alternating *Agrawal* concepts used to perform a sensitivity analysis on the cluster count hyperparameter (k).
 - *ConflictingContextStream* (Exp. 9): Features contradictory labeling logic tied to an explicit context feature, used in the ablation study to verify the necessity of the "Better-than-Random" deactivation rule.
 - **Standard Benchmarks:**

- *BenchmarkDriftStream* (Exp. 7): A 40,000-sample synthetic benchmark utilizing four sequential variants of the *SEA* generator with 10% noise.
- Real-World Datasets (Exp. 8): To evaluate the framework’s performance on real-world data subject to natural drifts, we include four standard stream mining benchmark datasets:
 - * *Phishing Website Detection*: A binary classification dataset (1,250 samples) aimed at identifying malicious URLs.
 - * *Electricity Pricing (Elec2)*: A dataset tracking fluctuating electricity prices in the New South Wales market to predict price changes.
 - * [*Forest Covertypes*: A dataset collected by the US Forest Service, predicting forest cover types based on cartographic and spatial variables.](#)
 - * [*Airlines*: A dataset containing commercial flight departure information within the US, aiming to predict flight delays based on scheduling and spatial data.](#)

3.2 Evaluation Protocol

All experiments follow a **prequential (test-then-train)** protocol. Every instance in the stream is first used to test the model’s predictive accuracy before being used for training.

Performance is monitored via **Rolling Accuracy** (using window sizes between 100 and 1,000 samples) to observe real-time adaptation, and **Cumulative Accuracy** for overall robustness. For imbalanced benchmarks, we report **Cohen’s Kappa**. Finally, we measure **Computational Latency** in milliseconds per sample (*ms/sample*) to quantify the overhead introduced by the C-DES clustering and selection logic.

4 Results and Discussion

The evaluation of the C-DES framework across ten distinct experimental scenarios provides a comprehensive overview of its performance in non-stationary environments. The following subsections detail the observations for each experiment. [Across the experimental plots, visual markers are used to distinguish drift severity: an **X** indicates a major structural drift detected at the ensemble level, while an **o** signifies a minor localized drift detected by an individual base learner.](#)

Updates Compared to the Preliminary Report

The experimental evaluation was expanded to include two new real-world datasets, and visual markers were introduced to the time-series plots to explicitly distinguish between drift detections at the base learner and ensemble levels.

Experiment 1: Regional Concept Drift. In this scenario, which involves localized changes in the *SEA* variants, the Base C-DES algorithm demonstrated a robust ability to maintain high accuracy in stable regions while effectively adapting to drifting regions, similar to the Vanilla SRP algorithm. As illustrated in Figures 1 and 2, there is no statistically significant difference in performance between the algorithms, with all four tested configurations maintaining a consistent accuracy of approximately 0.9 across both stable and drifting segments. This uniformity indicates that C-DES’s localized ensemble selection is inherently sufficient for managing this type of regional drift without requiring the additional resets provided by SDDM or ADWIN. Essentially, the dynamic weighting and continuous learning of base learners allow the system to recover naturally within the affected clusters.

Experiment 2: Contextual Sensor Drift. This experiment evaluated the framework’s response to inverted logic within a high-temperature context. While Base C-DES maintains superior accuracy in the low-temperature region throughout its lifetime, it initially underperforms compared to Vanilla SRP in the high-temperature region immediately following the drift. However, as shown in Figures 3 and 4, Base C-DES converges to a performance level comparable to Vanilla SRP after processing approximately 3,000 new samples. The results highlight a trade-off between stability and adaptation speed: the active adaptation in C-DES-Hybrid recovered more quickly in the drifting high-temperature region but exhibited lower accuracy in stable regions compared to the passive C-DES-SDDM configuration. This suggests that while model-level tree replacement accelerates recovery from severe contextual drift, it may lead to premature resets in stable clusters where continuous learning would have sufficed.

Experiment 3: Recurrent Drift ($A \rightarrow B \rightarrow A$). Results from the *Agrawal* recurrent stream highlight the "memory" capability of the ensemble. As shown in Figure 5, Base C-DES maintains higher accuracy throughout the majority of the training process, particularly after the stream reverts to concept A, suggesting superior pattern retention. The data shows that

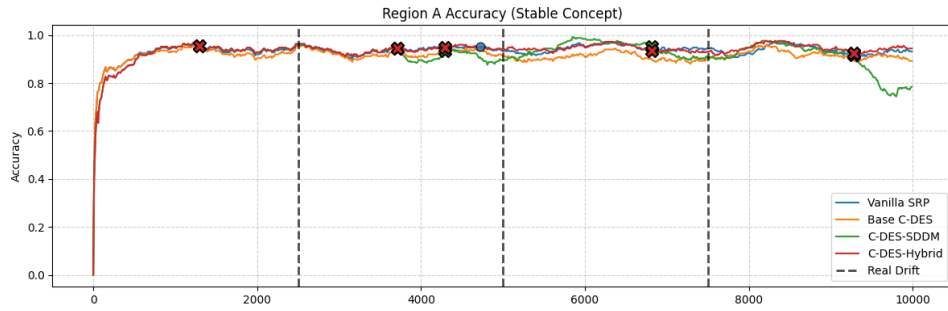


Figure 1: [Accuracy over time in the stable region, evaluated on the *RegionalDriftStream*.](#)

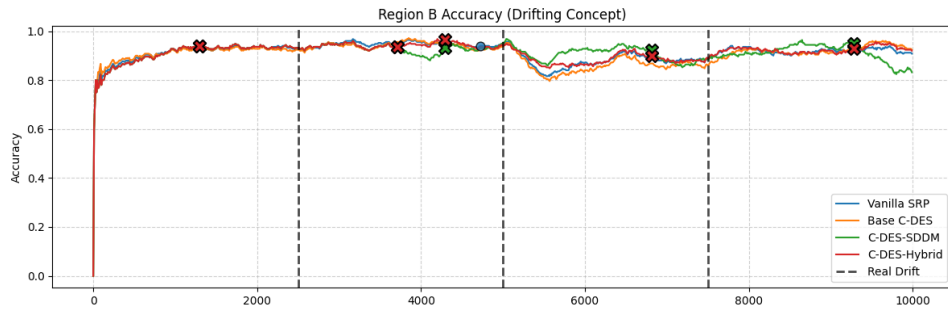


Figure 2: [Accuracy over time for the drifting region, evaluated on the *RegionalDriftStream*.](#)

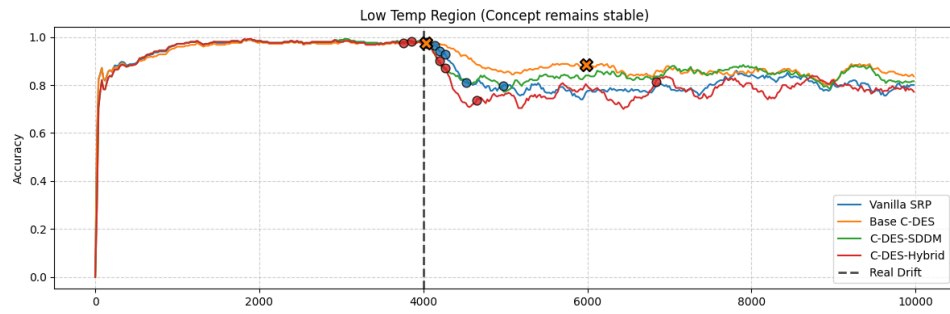


Figure 3: [Accuracy over time in the low temperature region, evaluated on the *SensorFailureStream*.](#)

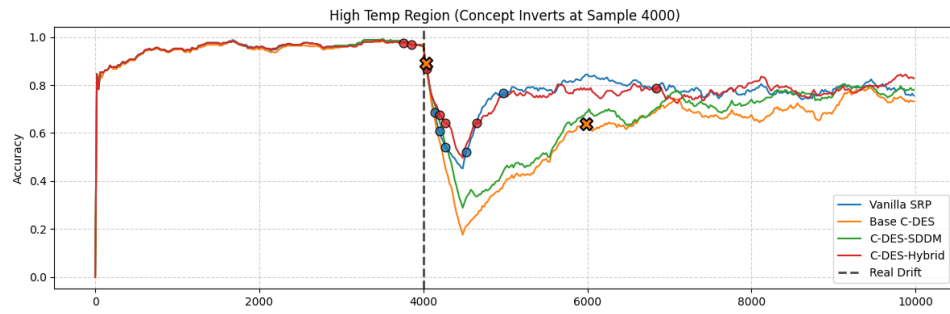


Figure 4: [Accuracy over time for the high temperature region.](#)

while C-DES-Hybrid offers the fastest initial recovery after a drift, its active replacement strategy can occasionally hinder performance in recurrent scenarios compared to the passive Base C-DES or C-DES-SDDM configurations. These passive approaches demonstrate stronger stability during recurring shifts, eventually adapting faster than Vanilla SRP as the underlying clusters effectively re-associate with previously established competencies.

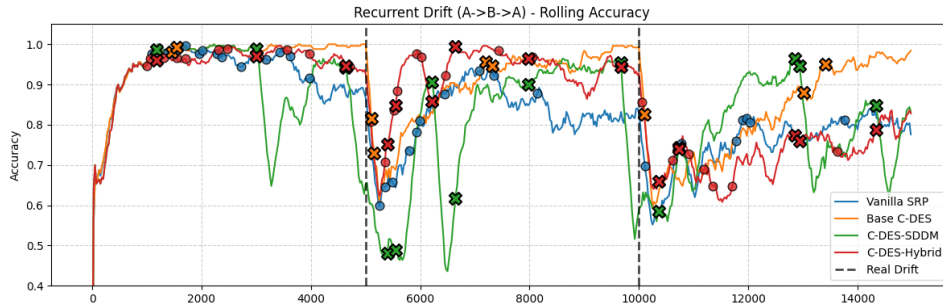


Figure 5: [Recurrent Drift Accuracy comparison, evaluated on the *RecurrentDriftStream*.](#)

Experiment 4: Rapid Concept Flickering. The stability of localized competency trackers was tested under high-frequency switching (every 250 samples). While both models exhibit nearly identical accuracy during the initial phase, Figure 6 demonstrates that Base C-DES eventually outperforms Vanilla SRP as the stream progresses, indicating better adaptation to volatile environments. The late-stage success of C-DES-SDDM and the standard Base C-DES suggests that passive adaptation is superior in flickering environments where rapid, "active" resets can become counterproductive. In this high-volatility context, the aggressive tree replacements triggered by ADWIN in both the Vanilla SRP and the C-DES-Hybrid configurations likely prevented the models from stabilizing, leading to the observed performance degradation after 2,500 samples.

Experiment 5: Hyperparameter Sensitivity (Cluster Count k). We analyzed the trade-off between granularity and computational overhead by varying the cluster count k from 2 to 20. Interestingly, the results in Figure 7 indicate that accuracy is not significantly affected by the number of clusters within the tested range, suggesting the model is robust to this specific hyperparameter.

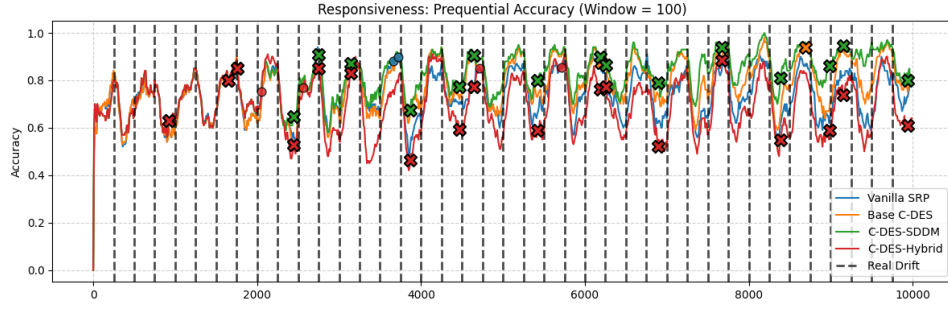


Figure 6: [Accuracy over time for high-frequency concept changes, evaluated on the *RapidFlickerStream*.](#)

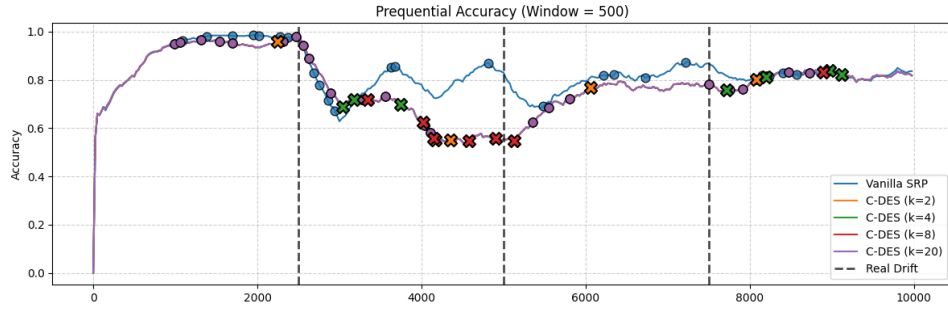


Figure 7: [Impact of cluster count \$k\$ on model accuracy, evaluated on the *MultiContextStream*.](#)

Experiment 6: The Curse of Dimensionality. By injecting up to 100 irrelevant noise features, we observed the operational limits of the KMeans context mapping. To ensure statistical reliability and capture performance variance, the experiment was repeated 20 times for each noise level configuration, with the resulting distributions plotted in Figure 8. The results demonstrate that the passive C-DES configurations (Base C-DES and C-DES-SDDM) exhibit superior resilience to low and moderate noise (0 to 10 injected features), maintaining notably higher median accuracies than Vanilla SRP. However, as the feature space expands to extreme dimensions (50 and 100 noise features), the distance calculations within the clustering layer degrade. Consequently, the localized advantage diminishes, causing all models to converge around a median accuracy of approximately 65%. Interestingly, while the active C-DES-Hybrid configuration starts with a lower baseline accuracy.

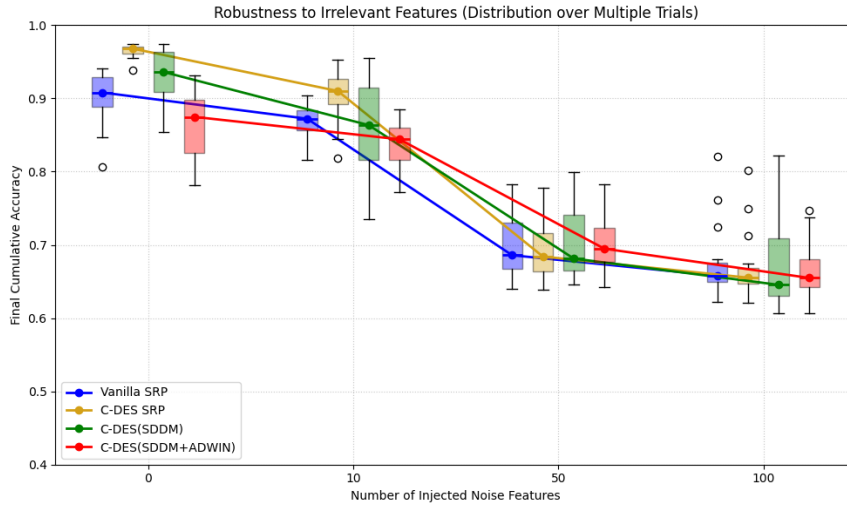


Figure 8: Performance degradation under increasing feature noise, evaluated on the *NoisyAgrawalStream*.

Experiment 7: Standard Synthetic Benchmarks (SEA). This long-form experiment (40,000 samples) provided a baseline for global drift handling. Table 1 summarizes the performance metrics. While Base C-DES shows a slight decrease in accuracy and higher latency compared to Vanilla SRP and ARF, it remains a competitive option for context-aware scenarios.

Table 1: Benchmark Results on the SEA Dataset.

Model	Accuracy (%)	Cohen’s Kappa	Latency (ms/sample)
HAT (Single Tree)	85.97%	0.6962	0.14
ARF (Ensemble)	87.41%	0.7272	1.29
Vanilla SRP	85.35%	0.6787	2.65
Base C-DES	83.45%	0.6371	3.24

Experiment 8: Real-World Benchmarks (Phishing and Elec2). [Testing on four real-world datasets \(Airlines, Electricity Pricing, Forest Covertypes, and Phishing Website Detection\)](#) showed how well the framework works on actual data rather than just artificial streams. As shown in Table 2, the C-DES models perform very similarly to the standard Vanilla SRP baseline. On the Phishing and Airlines datasets, the passive Base C-DES achieves almost the exact same accuracy as the baseline. On the more complex Forest Covertypes and Electricity datasets, the passive approach drops behind a little. However, the active C-DES-Hybrid model makes up for this gap, and even slightly beats Vanilla SRP on the Covertypes dataset (93.39% vs. 93.28%). This proves that our local selection method handles real-world data successfully when paired with the right drift reset strategy.

Experiment 9: The Quarantine Effect (Ensemble Churn). The final experiment measured the frequency of drift detections triggered by localized noise. Observations from Figure 9 suggest that Base C-DES performs comparably with the baseline in managing ensemble churn.

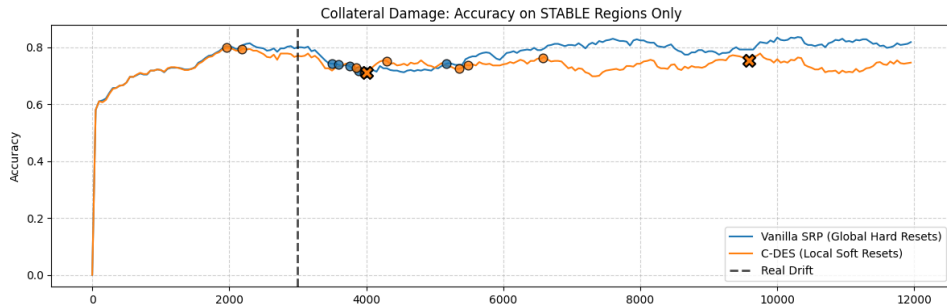


Figure 9: [Ensemble churn and drift detection frequency, evaluated on the NoiseQuarantineStream.](#)

Table 2: Real-World Benchmark Performance, evaluated on Phishing, Elec2, Coverttype, and Airlines streams.

Dataset	Model	Final Accuracy
Airlines (Spatial/Geo)	Vanilla SRP	66.76%
Airlines (Spatial/Geo)	C-DES-Hybrid	64.93%
Airlines (Spatial/Geo)	C-DES-SDDM	63.88%
Airlines (Spatial/Geo)	Base C-DES	66.30%
Electricity Pricing	Vanilla SRP	87.96%
Electricity Pricing	C-DES-Hybrid	87.46%
Electricity Pricing	C-DES-SDDM	85.53%
Electricity Pricing	Base C-DES	84.59%
Forest Coverttype	Vanilla SRP	93.28%
Forest Coverttype	C-DES-Hybrid	93.39%
Forest Coverttype	C-DES-SDDM	92.21%
Forest Coverttype	Base C-DES	86.14%
Phishing Website Detection	Vanilla SRP	91.19%
Phishing Website Detection	C-DES-Hybrid	91.11%
Phishing Website Detection	C-DES-SDDM	91.11%
Phishing Website Detection	Base C-DES	91.19%

5 Conclusion

This work presented a novel framework for data stream classification that integrates Cluster-Based Dynamic Ensemble Selection (C-DES) with the Statistical Drift Detection Method (SDDM). By shifting the focus from global drift handling to a localized, region-specific approach, we developed a system capable of managing non-stationary environments with high precision. Our architecture specifically enabled the comparative analysis of two primary drift-handling strategies: Passive Adaptation, which relies on SDDM to trigger soft resets of ensemble weights, and Active Adaptation, which supplements this with ADWIN for aggressive, model-level tree replacement.

The experimental evaluation revealed a fundamental trade-off between adaptation speed and ensemble stability. We found that the core localized selection mechanism of C-DES is highly effective at isolating concept drifts to specific regions of the feature space. When confronted with severe contextual changes, the Active Adaptation strategy facilitated significantly faster recovery times. However, this aggressive model replacement introduced notable instability in highly volatile or recurrent environments. Conversely, the Passive Adaptation strategy proved superior in maintaining long-term stability and pattern retention, allowing the ensemble to effectively recover and re-exploit past concepts without destroying historical knowledge.

Furthermore, the framework demonstrated significant robustness to high-dimensional noise and variations in hyperparameters, such as the cluster count, while remaining computationally competitive with state-of-the-art global ensembles. The dynamic deactivation rule was also validated as a critical component, successfully quarantining incompetent models during localized shifts. Ultimately, this research concludes that for environments characterized by recurring or flickering changes, the passive resetting of localized competency metrics offer the optimal balance of stability and performance, whereas active adaptation is best reserved for severe, permanent conceptual shifts.

6 Repository and Division of Work

The project repository is available at: [GitHub](#)

Author	Contribution
Michał Iwicki	Development of the SDDM method and adaptation of the C-DES framework to integrate SDDM
Dawid Sroczyk	Design and setup of the experimental evaluation
Marta Szuwarska	Implementation of the C-DES framework
All Members	Justified stream mining necessity, selected baseline algorithms, and collectively analyzed experimental results

References

- [1] Simona Micevska, Ahmed Awad, and Sherif Sakr. “SDDM: an interpretable statistical concept drift detection method for data streams”. In: *Journal of Intelligent Information Systems* 56 (2021), pp. 459–484. DOI: 10.1007/s10844-020-00634-5.
- [2] Ahmed H. Madkour, Hatem M. Abdelkader, and Amgad M. Mohammed. “Dynamic Classification Ensembles for Handling Imbalanced Multiclass Drifted Data Streams”. In: *Information Sciences* 670 (2024), p. 120555. ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2024.120555>. URL: <https://www.sciencedirect.com/science/article/pii/S0020025524004687>.